HW3

November 7, 2026

1 Stats 21 - HW 3

2 Sam Kim (406120141)

Homework copyright Miles Chen. The word problems are adapted from exercises in *Think Python*, 2nd edition, by Allen B. Downey.

Complete the code and written responses. Before submitting, restart the kernel and run your completed notebook from top to bottom. Make sure all requested results are visible. Either `print(...)` or a visible notebook result is acceptable.

Export to HTML, print to a letter-sized PDF, and submit the PDF to Gradescope. Assign/tag the pages to the corresponding questions. Follow the course submission instructions for the completed notebook file as well.

2.1 Academic Integrity

The homework is practice for building your own understanding. Work through the problems, check your reasoning, and use permitted help to support your learning.

Replace `[Your Name]` to acknowledge the following:

> I, Sam Kim, agree to follow the course academic integrity policies and the Stats 21 AI and Collaboration Policy. My submitted work reflects my own thinking and understanding, and I can explain my code and results.

Acknowledgments and AI disclosure: Give specific credit for people or sources you relied on, and disclose any AI assistance, including hints, debugging, or editing. Briefly state which problems and how each helped, or write "None".

> *Extensively referenced the NumPy docs.* In particular, these articles:
>
> - np.normal()
> - np.round()
> - np.absolute()
> - np.reshape()
> - np.isin()
> - np.transpose()
> - np.isclose()

2.2 Reading and class connections

Use *Think Python*, **3rd edition**, for the reading:

- Chapter 10: Dictionaries.
- Chapter 11: Tuples.
- Chapter 13: Files and Databases, focusing on text-file reading and writing; databases are not required here.

The exercise numbers and source links below refer to the **2nd edition**. The full prompts are included, so you do not need a separate copy of that edition.

Review lectures 8–10 for dictionaries, tuples, and files, and lectures 11–12 for NumPy arrays, copies/views, broadcasting, and Boolean masks.

2.3 Working on this homework

- Return the requested result from each function. Display results in the supplied test cells after completing the function. Keep the tests and add cases where requested.
- You may assume inputs meet the stated requirements; extra input-validation code is not required. Use clear intermediate steps; one-line solutions earn no extra credit.
- Use a dictionary for the word problems and NumPy array operations for the NumPy section. A solution should show the requested method, not hard-code the displayed output.
- Written checks normally need only one or two sentences.

`words.txt` is included beside this notebook in `homework/HW3`; it is the same word list used in HW2. Run this notebook with `HW3` as the working directory, or adjust the file path if you keep it elsewhere. Do not print the entire word dictionary.

2.4 Exercise 11.1: A dictionary of words

Adapted from Think Python 2e, Exercise 11.1.

Complete `make_word_dict(filename="words.txt")`. Read the file and **return** a dictionary whose keys are the words, with surrounding whitespace removed. Skip blank lines. You may use `None` for every value. The file contains lowercase words.

Save the returned dictionary as `word_dict`. Display its number of keys and the results of the membership checks below.

Understanding: Does `"hello" in word_dict` search keys or values? Why should we remove the newline before using a word as a key?

[1]:
```
def make_word_dict(filename="words.txt"):
    word_dict = {}
    with open(filename, "r") as f:
        for line in f:
            word_dict[line.strip()] = None
    return word_dict
```

[2]:
```
# Run after completing make_word_dict().
word_dict = make_word_dict()
```

```
print("Number of words:", len(word_dict))
print("hello" in word_dict)       # Expected: True
print("hello\n" in word_dict)   # Expected: False
print("" in word_dict)            # Expected: False
```

```
Number of words: 113776
True
False
False
```

Your explanation: The boolean expression must search the keys. We trivially know that if it searched the values, it'd return `False` since the values are all made of `None` values (as per the instructions). We can also intuitively derive this, since it is quite rare that one would look up a definition of a word in an actual dictionary rather than look up the word for its definition.

As for removing the newline character before using the words as keys, we must do so otherwise the key will be stored literally. In other words, if we search for, say, "hello" in the dictionary, we'd have to search `"hello\n"` if we kept the newline.

2.5 Exercise 11.4: Detecting duplicates with a dictionary

Adapted from Think Python 2e, Exercise 11.4.

Rewrite HW2's `has_duplicates(t)` using a dictionary to keep track of values already encountered. Return `True` if any value repeats, and `False` otherwise. Do not modify the input list, sort it, or replace the dictionary with a set.

As in HW2, inputs may mix integers and strings and will not contain nested lists. All values used here are hashable and can be dictionary keys. An empty list has no duplicates.

Understanding: What information does your dictionary represent as you traverse the list, and why can you return as soon as you find a repeat? No timing experiment or claim of a fixed speedup is required.

[3]:
```
# already beat you to it! this was my hw2 answer:

def has_duplicates(t):
    d = {}
    for i in t:
        if i in d.keys():
            return True
        else:
            d[i] = 1
    return False
```

[4]:
```
print(has_duplicates(['a', 'b', 'b', 'c']))  # Expected: True
print(has_duplicates(['a', 'b', 'c', 'a']))  # Expected: True
print(has_duplicates(['a', 'b', 'c']))       # Expected: False
print(has_duplicates([]))                   # Expected: False
print(has_duplicates([1, '1', 2]))           # Expected: False
```

```
values = [3, 1, 3]
before = values.copy()
print(has_duplicates(values))               # Expected: True
print(values == before)                     # Expected: True
# Add one more test of your own and state its expected result.
print(has_duplicates([5, "5"]))  # Expected: False
```

```
True
True
False
False
False
True
True
False
```

Your explanation: The keys represent the objects in the list. Each corresponding and respective value represents its existence. In other words, if a key-value pair is in the dictionary, it has been seen already (and thus exists) in the list. If, while scanning through the list, an object that has already been seen is scanned again, it will pass `i in d.keys()`. This means that the object has been seen before and this current `i` is a duplicate, leading to short-circuting.

2.6 Exercise 11.5: Caesar-cipher rotate pairs

Adapted from Think Python 2e, Exercise 11.5.

A Caesar rotation shifts each letter forward through the alphabet, wrapping from `z` to `a`. For example, `"cheer"` rotated by 7 becomes `"jolly"`.

1. Complete `rotate_word(word, shift)` for lowercase `a`–`z` words and integer shifts. Return a string. A shift of 26 leaves a word unchanged.
2. Complete `find_rotate_pairs(word_dict, min_length=5)`. Use the dictionary keys as the available words. Look for **distinct** words of at least `min_length` letters related by shifts 1 through 25.
3. Return a sorted list of tuples `(word1, word2, shift)`. Report each unordered pair once, with `word1 < word2` alphabetically. `shift` must rotate `word1` into `word2`. Do not also report the reverse pair or a word paired with itself.
4. Search the full word dictionary and display the **total number of pairs**. Then display only the **first 10 pairs from the sorted result**, one per line. Compute and return all pairs; limit only the displayed examples.

Hints: `ord()` converts a character to its numeric code and `chr()` converts a code back to a character. Modulo 26 can handle wraparound. Generate rotations and use dictionary membership instead of comparing every word with every other word.

Understanding: Why are shifts 1–25 sufficient for finding distinct rotate pairs, and how does your output rule avoid counting a pair twice?

[5]:
```
import copy

def rotate_word(word, shift):
    rotated = ""
    letters = [chr(x) for x in range(97, 123)]
    for c in word:
        cipher = letters[(ord(c) - 97 + shift) % 26]
        rotated += cipher
    return rotated

def find_rotate_pairs(word_dict, min_length=5):
    ret_dict = {}
    for w in word_dict:
        if len(w) < min_length: # skips all words under minimum word length
            continue
        for i in range(1, 26): # to iterate through all valid shifts from 1 to␣
 ↪25.
            query = rotate_word(w, i)
            if query in word_dict.keys() and not(query in ret_dict.keys()):
                # checks if rotated word is valid AND NOT already in a pair␣
 ↪found earlier
                # -----
                # this works since this function goes through the word_dict␣
 ↪alphabetically
                # so any word's rotation will be "greater" than itself
                # (shift is at least 1 and at most 25). the rotation, as it is␣
 ↪valid, will
                # inevitably be found in the dictionary and can be rotated back.
 ↪ however,
                # this "back-rotation" is already in the dictionary to be␣
 ↪returned and
                # thus will be skipped, preventing duplicate pairs.
                ret_dict[w] = (w, query, i)
    print(f'{len(ret_dict.keys())} pair(s) were found.')
    return list(ret_dict.values())
```

[6]:
```
print(rotate_word("cheer", 7))  # Expected: jolly
print(rotate_word("zoo", 1))    # Expected: app
print(rotate_word("abc", 26))   # Expected: abc

small_words = {"cheer": None, "jolly": None, "melon": None, "a": None}
print(find_rotate_pairs(small_words))  # Expected: [('cheer', 'jolly', 7)]
# Add one more test of your own, including its expected result.
a_word = {"quay": None, "wage": None}
print(find_rotate_pairs(a_word, min_length=1))
```

```
jolly
```

```
app
abc
1 pair(s) were found.
[('cheer', 'jolly', 7)]
1 pair(s) were found.
[('quay', 'wage', 6)]
```

[7]:
```
# Find all pairs from word_dict; display the total count and the first 10␣
↪sorted pairs.
```

[8]:
```
print(find_rotate_pairs(word_dict, min_length=0)[0:11]) # entire word_dict␣
↪implies all valid pairs, regardless of length.
print(find_rotate_pairs(word_dict, min_length=5)[0:11])
```

```
861 pair(s) were found.
[('aah', 'eel', 4), ('aba', 'tut', 19), ('abet', 'hila', 7), ('abjurer',
'nowhere', 13), ('abo', 'nob', 13), ('aby', 'zax', 25), ('ache', 'gink', 6),
('act', 'yar', 24), ('acts', 'sulk', 18), ('ad', 'or', 14), ('add', 'loo', 11)]
97 pair(s) were found.
[('abjurer', 'nowhere', 13), ('adder', 'beefs', 1), ('ahull', 'gnarr', 6),
('alkyd', 'epoch', 4), ('alula', 'tenet', 19), ('ambit', 'setal', 18),
('anteed', 'bouffe', 1), ('aptly', 'timer', 19), ('arena', 'river', 17),
('baiza', 'tsars', 18), ('banjo', 'ferns', 4)]
```

Your explanation: A range of 1 to 25 is enough as 1 guarantees a shift but 26 mod 26 = 0, so you'd just get the word itself. In other words, the alphabet has 26 letters, so you'd just cycle around. As for an explanation of the output rule, please see the verbose embedded comment.

2.7 Exercise 12.2: Grouping anagrams

Adapted from Think Python 2e, Exercise 12.2.

Anagrams use exactly the same letters with the same multiplicities. Complete `group_anagrams(words)` for an iterable of distinct lowercase words. Return a dictionary that maps a **tuple of sorted letters** to a list of its words. For example, `"eat"` and `"tea"` share the key `('a', 'e', 't')`.

Use the keys of `word_dict` as input. Identify all groups containing **at least six distinct words** and display the **total number of qualifying groups**. Alphabetize the words within each qualifying group, then order the groups by their first word. Display only the **first 10 groups** in that order, one per line. Compute all groups before limiting the display. Groups with fewer than six words should remain in the returned dictionary but need not be printed.

Example of the required group display:

```
['abets', 'baste', 'bates', 'beast', 'beats', 'betas', 'tabes']
```

Understanding: Why can a tuple of letters be a dictionary key while the list returned by `sorted(word)` cannot? Why must the key retain repeated letters, as in `"abb"` versus `"ab"`?

[9]:
```
def group_anagrams(words):
    ret_dict = {}
    for w in words:
        constituents = tuple(sorted(w))
        if constituents not in ret_dict.keys():
            ret_dict[constituents] = [w]
        else:
            words = ret_dict[constituents]
            words.append(w)
            ret_dict[constituents] = words
    return ret_dict
```

[10]:
```
small_groups = group_anagrams(["eat", "tea", "ate", "bat", "tab", "abb", "ab"])
print(sorted(small_groups[('a', 'e', 't')]))  # Expected: ['ate', 'eat', 'tea']
print(sorted(small_groups[('a', 'b', 't')]))  # Expected: ['bat', 'tab']
print(small_groups[('a', 'b', 'b')])          # Expected: ['abb']
print(small_groups[('a', 'b')])               # Expected: ['ab']
```

```
['ate', 'eat', 'tea']
['bat', 'tab']
['abb']
['ab']
```

[11]:
```
# Group the full word dictionary; display the total qualifying-group count
# and the first 10 qualifying groups in the order specified above.
groups = {k: v for k, v in group_anagrams(word_dict).items() if len(v) >= 6}
l = list(groups.keys())[0:10]

print(f"Number of qualifying pairs: {len(groups.keys())}", [groups[x] for x in␣
↪l])
```

```
Number of qualifying pairs: 104 [['abets', 'baste', 'bates', 'beast', 'beats',
'betas', 'tabes'], ['acers', 'acres', 'cares', 'carse', 'escar', 'races',
'scare', 'serac'], ['acred', 'arced', 'cadre', 'cared', 'cedar', 'raced'],
['aiders', 'deairs', 'irades', 'raised', 'redias', 'resaid'], ['airts', 'astir',
'sitar', 'stair', 'stria', 'tarsi'], ['alert', 'alter', 'artel', 'later',
'ratel', 'taler'], ['alerting', 'altering', 'integral', 'relating', 'tanglier',
'triangle'], ['alerts', 'alters', 'artels', 'estral', 'laster', 'ratels',
'salter', 'slater', 'staler', 'stelar', 'talers'], ['alevin', 'alvine',
'valine', 'veinal', 'venial', 'vineal'], ['algins', 'aligns', 'lasing',
'liangs', 'ligans', 'lingas', 'signal']]
```

Your explanation: Unlike lists, tuples are immutable. You cannot hash a mutable object, so you can't use lists as dictionary keys. These keys must also retain repeated letters lest you return words that are not actually anagrams in the same group, even though they share the same letters: e.g. "ban" and "banana." Letter counts matter.

2.8 NumPy exercises

Display each requested result. Use array operations and Boolean masks rather than Python loops for these exercises. Functions not demonstrated in class have documentation links below; looking up an API is part of the assignment.

Use the supplied random-number generators. A generator advances when it draws values: restart its setup cell before repeating an exercise if you want the same draws. Exact streams are not guaranteed across NumPy versions; correctness also depends on the requested shapes, ranges, and calculations, not only printed digits.

For NP.2b and NP.7, explicitly use `ddof=0` for standard deviation, as specified in those questions. Keep full precision for calculations unless rounding is requested.

[12]:
```
import numpy as np
```

2.8.1 NP.1a: Random integers

Using the generator below, create `b`, a one-dimensional array of 10 random integers from **0 through 99 inclusive**, with repetitions allowed. Display it.

In the course environment, the first draw gives:

```
[47 51 75 95  3 14 82 94 24 31]
```

[13]:
```
rng = np.random.default_rng(seed=1)
```

[14]:
```
b = np.array(np.random.randint(low=0, high=100, size=10))
print(b)
```

```
[38 90 63  3 85 32 81  6 30  3]
```

2.8.2 NP.1b: Reverse a slice

Use slicing with a negative step to create `b_reversed`, containing the elements of `b` in reverse order. Display both arrays without changing their values.

Understanding: Does this basic slice share data with `b`? If you assigned to an element of `b_reversed`, would `b` change? Explain using the view/copy distinction from lecture 11; you do not need to make that assignment.

[15]:
```
# Create and display b_reversed, then display b.
b_reversed = b[::-1]
print(b, b_reversed, sep='\n')

b_reversed[5] = 99999
print(b, b_reversed, sep='\n')
```

```
[38 90 63  3 85 32 81  6 30  3]
[ 3 30  6 81 32 85  3 63 90 38]
[   38    90    63     3 99999    32    81     6    30     3]
[    3    30     6    81    32 99999     3    63    90    38]
```

Your explanation: Performing a slice on a NumPy array simply returns a view, which shares the data with the original. Indeed, through the assignment above, we can see that it does modify the original `b` array. In order for this to not happen, we'd need to have a `.copy()` of `b`.

2.8.3 NP.2a: Simulate normal observations

Using the fresh generator below, draw 1,000 values from a normal distribution with **population mean 100 and population standard deviation 15**. Round the values to one decimal place and store the resulting array as `c`.

Display only the first 10 values of `c`, followed by its length, mean, and standard deviation (`ddof=0`). Do not print all 1,000 values.

Understanding: Why need the observed mean and SD not equal 100 and 15 exactly?

[16]:
```
rng = np.random.default_rng(seed=2)
```

[17]:
```
# generates 1000 values under N(100, 15), rounds each to 1 dec. point, then␣
 ↪creates an array out of it.
c = np.array(np.round(np.random.normal(100, 15, size=1000), decimals=1))

print(c[0:10], len(c), np.mean(c), np.std(c, ddof=0), sep="\n")
```

```
[ 90.8 102.3 105.3  84.   81.7 106.2  92.4 114.8  91.8  99.7]
1000
100.6276
14.92455621584776
```

Your explanation: These 1,000 values represent a random sample of a population whose mean and SD are 100 and 15 respectively. As such, there will be sampling variation. As the sample size increases, however, we should expect the mean and SD of the sample to more closely represent the population's.

2.8.4 NP.2b: A Boolean mask for extreme values

Using the rounded array `c`, compute its observed mean and SD with `ddof=0`. Select values whose **absolute distance from that mean is strictly greater than three times that SD**. Use the observed quantities, not the generating parameters 100 and 15.

Display the selected values, their count, their proportion of all 1,000 values, and the corresponding percentage. For example, 3 out of 1,000 would be a proportion of `0.003`, or `0.3%`—not a proportion of `0.3`.

Understanding: Why does taking the mean of a Boolean mask give the proportion selected? There is no requirement that this simulated sample have a particular number of extreme values.

[18]:
```
# Compute the mask and display the values, count, proportion, and percentage.
c_mean = np.mean(c)
c_std = np.std(c, ddof=0)

c_outliers = c[(np.absolute(c - c_mean)) > (3 * c_std)]
```

```
print(c_outliers, f"{len(c_outliers)} outliers exist, making {len(c_outliers)/
  ↪len(c) * 100}% of the sample of size {len(c)}.", sep="\n")
```

```
[146.6]
1 outliers exist, making 0.1% of the sample of size 1000.
```

Your explanation: This would give the proper proportion as we are dividing the number of elements that satisfy the mask by the total number of elements. In this case, we would get the correct number of outliers — a drawn-out approach can be seen in my `print()` function

2.8.5 NP.3: An identity matrix

Create and display a 3×3 identity array named `I3`: ones on the main diagonal and zeros elsewhere. Its shape should be `(3, 3)`.

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

[19]:
```
# Create and display I3.
I3 = np.identity(3)
print(I3)
```

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

2.8.6 NP.4a: Reshape a sequence

Create `X`, a 10×10 array containing the integers 1 through 100, arranged across rows in increasing order. Display the full array and its shape. The first row should contain 1–10 and the last row 91–100.

[20]:
```
# Create and display X and its shape.
X = np.reshape(np.arange(1, 101), (10, 10))
print(X, X.shape, sep="\n")
```

```
[[  1   2   3   4   5   6   7   8   9  10]
 [ 11  12  13  14  15  16  17  18  19  20]
 [ 21  22  23  24  25  26  27  28  29  30]
 [ 31  32  33  34  35  36  37  38  39  40]
 [ 41  42  43  44  45  46  47  48  49  50]
 [ 51  52  53  54  55  56  57  58  59  60]
 [ 61  62  63  64  65  66  67  68  69  70]
 [ 71  72  73  74  75  76  77  78  79  80]
 [ 81  82  83  84  85  86  87  88  89  90]
 [ 91  92  93  94  95  96  97  98  99 100]]
(10, 10)
```

2.8.7 NP.4b: Copy and mask

Make an **independent copy** of `X` named `Y`. Create an array of the ten perfect squares between 1 and 100, then use a Boolean mask to replace every nonsquare in `Y` with zero. Do not change `X`. You may prepare the squares and mask in separate lines; there is no one-line requirement.

See `np.isin` for an elementwise membership test. Display the full `Y`, and display `X` to confirm it is unchanged. `Y` should retain exactly 10 nonzero entries.

Understanding: Why would `Y = X` fail to provide an independent copy?

[21]:
```
# Copy X, replace nonsquares in Y, and display the requested results.
Y = X.copy()

squares = np.array(np.arange(1, 11) ** 2)
Y[np.isin(Y, squares, invert=True)] = 0

print(f"Y: {Y}\n\nX: {X}")
```

```
Y: [[  1   0   0   4   0   0   0   0   9   0]
 [  0   0   0   0   0  16   0   0   0   0]
 [  0   0   0   0  25   0   0   0   0   0]
 [  0   0   0   0   0  36   0   0   0   0]
 [  0   0   0   0   0   0   0   0  49   0]
 [  0   0   0   0   0   0   0   0   0   0]
 [  0   0   0  64   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0]
 [ 81   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0 100]]

X: [[  1   2   3   4   5   6   7   8   9  10]
 [ 11  12  13  14  15  16  17  18  19  20]
 [ 21  22  23  24  25  26  27  28  29  30]
 [ 31  32  33  34  35  36  37  38  39  40]
 [ 41  42  43  44  45  46  47  48  49  50]
 [ 51  52  53  54  55  56  57  58  59  60]
 [ 61  62  63  64  65  66  67  68  69  70]
 [ 71  72  73  74  75  76  77  78  79  80]
 [ 81  82  83  84  85  86  87  88  89  90]
 [ 91  92  93  94  95  96  97  98  99 100]]
```

Your explanation: Assignment does not do a deep copy. It's akin to telling someone the address of some place — i.e. Y would hold X's address rather than its own building identical to X's.

2.8.8 NP.5: Tile a checkerboard

Start with the integer array `[[1, 0], [0, 1]]`. Use `np.tile` to repeat it into an $\mathbf{8} \times \mathbf{8}$ array named `checkers`. Display it and its shape.

The first row should be `[1 0 1 0 1 0 1 0]` and the second `[0 1 0 1 0 1 0 1]`, with those rows alternating.

[22]:
```
# Create and display checkers and its shape.
checkers = np.tile([[1, 0], [0, 1]], (4, 4))
print(checkers, checkers.shape, sep="\n")
```

```
[[1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]]
(8, 8)
```

2.8.9 NP.6: An elementwise unit conversion

Convert every value of `f_temp` from Fahrenheit to Celsius: subtract 32, then multiply by 5/9. Round to one decimal place and save the result as `c_temp`. Display it without modifying `f_temp`.

The result retains shape `(5, 6)`. Its first value should be `21.1` and its last value `37.2`.

[23]:
```
# Supplied input; do not modify this cell.
f_temp = np.arange(70, 100).reshape((5, 6))
```

[24]:
```
# Create and display c_temp.
c_temp = ((f_temp - 32) * 5 / 9).round(decimals=1)
print(c_temp, c_temp.shape, sep='\n')
```

```
[[21.1 21.7 22.2 22.8 23.3 23.9]
 [24.4 25.  25.6 26.1 26.7 27.2]
 [27.8 28.3 28.9 29.4 30.  30.6]
 [31.1 31.7 32.2 32.8 33.3 33.9]
 [34.4 35.  35.6 36.1 36.7 37.2]]
(5, 6)
```

2.8.10 NP.7: Standardize each column

Convert the supplied array `x` to columnwise z-scores, stored in `z`: subtract each column's mean and divide by that column's SD. Use **`ddof=0`** for both standardizing and checking the result. These columns all have nonzero SD.

Use broadcasting, without loops. Display `z`, its column means, and its column SDs. The means should be approximately zero and the SDs approximately one. Do not round `z` before checking; values such as `1e-16` can be floating-point rounding rather than an error.

Understanding: Which `axis` produces one summary per column, and what is the shape of that summary? Explain how it broadcasts against `x` of shape `(10, 3)`.

[25]:
```
# Supplied input; do not modify this cell.
rng = np.random.default_rng(seed=100)
```

```
x = rng.integers(0, 100, size=30).reshape(10, 3)
```

[26]:
```
# Create z and display it, its column means, and its column SDs.
z = (x - x.mean(axis=0)) / x.std(axis=0, ddof=0)
print(z,
      z.mean(axis=0),
      # f"Checking if each column main is close to 0: {np.isclose(z.
 ↪mean(axis=0), 0)}",
      z.std(axis=0, ddof=0),
sep='\n')
```

```
[[ 0.41148428  1.04307072 -1.43031951]
 [-0.34069129 -1.14825432 -0.77270135]
 [-1.00437562 -1.26512499  0.46033272]
 [ 1.34064235  1.36446505  0.50143385]
 [-1.13711249  0.92620005  1.85777132]
 [ 1.07516861 -1.38199565  0.87134407]
 [ 0.19025617 -0.85607765  0.13152363]
 [ 1.38488797  0.60480571 -0.77270135]
 [-1.53532308  0.42949971 -1.34811724]
 [-0.38493691  0.28341137  0.50143385]]
[-1.16573418e-16  7.77156117e-17  1.44328993e-16]
[1. 1. 1.]
```

Your explanation: `axis=0`. The shape is `(3, )` for each summary, which makes sense as there are 3 columns in x and it reduces the 10 rows.

2.8.11 NP.8: Scale each column to 0–10

For the supplied `x`, linearly rescale **each column separately** so that its minimum becomes 0 and its maximum becomes 10. Round to two decimal places and store the result in `y`. Display `y` and its column minima and maxima.

These columns all have different minima and maxima. In the course environment, the first two rows are:

```
[[6.67 8.83 0.  ]
 [4.09 0.85 2.  ]]
```

Understanding: How is this transformation different from z-score standardization? What would happen to your formula if a column were constant? You do not need to implement a special rule for constant columns.

[27]:
```
# Supplied input; do not modify this cell. Same values as NP.7.
rng = np.random.default_rng(seed=100)
x = rng.integers(0, 100, size=30).reshape(10, 3)
print(x)
```

```
[[76 83 12]
 [59  8 28]
 [44  4 58]
```

```
 [97 94 59]
 [41 79 92]
 [91  0 68]
 [71 18 50]
 [98 68 28]
 [32 62 14]
 [58 57 59]]
```

[28]:
```
# Create y and display it, its column minima, and its column maxima.
y = ((x - x.min(axis=0)) / (x.max(axis=0) - x.min(axis=0)) * 10).round(2)
print(y, y.min(axis=0), y.max(axis=0), sep='\n')
```

```
[[ 6.67  8.83  0.  ]
 [ 4.09  0.85  2.  ]
 [ 1.82  0.43  5.75]
 [ 9.85 10.    5.88]
 [ 1.36  8.4  10.  ]
 [ 8.94  0.    7.  ]
 [ 5.91  1.91  4.75]
 [10.    7.23  2.  ]
 [ 0.    6.6   0.25]
 [ 3.94  6.06  5.88]]
[0. 0. 0.]
[10. 10. 10.]
```

Your explanation: With z-score standardization, we essentially see how far each data point is from the mean. We also make the mean 0 and standard deviation 1 this way. With normalization, we see how far each data point is from the minimum. If a column were to be a constant, the max and min values would be identical, so you'd be dividing by 0.

2.8.12 NP.9: Detect and replace NaN values

Make an independent copy of the supplied `x` named `x_filled`. Use a Boolean mask to replace its `NaN` entries with zero, leaving the original `x` unchanged. Display the original array, the filled array, and the number of entries replaced.

Expected filled array:

```
[[76. 83. 12.]
 [59.  8.  0.]
 [44.  0. 58.]
 [ 0. 94. 59.]]
```

Understanding: Why should you use `np.isnan(x)` instead of `x == np.nan`? Replacing missing values with zero here is an array-manipulation exercise. Give one reason it could distort an analysis if zero were not a justified value for a missing measurement.

[29]:
```
# Supplied input; do not modify this cell.
rng = np.random.default_rng(seed=100)
x = rng.integers(0, 100, size=12).reshape(4, 3).astype(float)
```

```
row = np.array([1, 2, 3])
col = np.array([2, 1, 0])
x[row, col] = np.nan
```

[30]:
```
# Create x_filled, replace its NaNs, and display the requested results.
x_filled = x.copy()
x_filled[np.isnan(x)] = 0
print(x)
print(x_filled)
print(f'Number of NaN entries replaced: {np.isnan(x).sum()}')
print(x == np.nan) # inherently incorrect; there should at least be 3 True values
```

```
[[76. 83. 12.]
 [59.  8. nan]
 [44. nan 58.]
 [nan 94. 59.]]
[[76. 83. 12.]
 [59.  8.  0.]
 [44.  0. 58.]
 [ 0. 94. 59.]]
Number of NaN entries replaced: 3
[[False False False]
 [False False False]
 [False False False]
 [False False False]]
```

Your explanation: `NaN` is not equal to itself. If any element in `x` were to equal `NaN`, `x == np.nan` would still return `False`.